

AI LEARNING EBOOK

AI, ML, Deep Learning, Agentic AI & Enterprise Engineering

A project-first roadmap from beginner foundations to production-grade enterprise AI systems, generated from your roadmap and expanded with missing real-world engineering layers.

7

Tracks

28

Modules

30+

Projects

1

Final platform

Table of Contents

1. Executive Learning Map
2. Recommended Timeline
3. Foundation
4. Machine Learning
5. Deep Learning
6. LLMs & Transformers
7. Agentic AI
8. Enterprise & MLOps
9. Missing Enterprise Foundations Added
10. Final eBook Project: AutoResearch AI
11. Assessment Framework
12. Project Artifact Template
13. Portfolio Checklist
14. References

OVERVIEW

Beginner to Enterprise AI Engineer Path

This eBook converts the interactive roadmap into a printable, project-first curriculum. Every major topic has a project, and the final project combines data engineering, ML, DL, LLMs, RAG, agents, MLOps, observability and governance.

7

Learning Tracks

28

Project Modules

12-18
months

Suggested Duration

1

Enterprise Capstone

● Beginner

● Intermediate

● Advanced

● Enterprise

● Capstone

How to use this eBook: finish each module by shipping the capstone project, not by only watching videos. Keep each project in GitHub with README, screenshots, metrics, and a short architecture note.

ROADMAP

Recommended Timeline

Use this as a serious beginner-to-enterprise plan. At 10-12 focused hours per week, expect 12-18 months. At 20+ hours per week, a disciplined learner can compress it to 8-10 months.

1

MONTHS 1-2

Python, Math, NumPy, pandas, SQL

Foundation

2

MONTHS 3-4

Classical ML, statistics, sklearn pipelines

Machine Learning

3

MONTHS 5-7

PyTorch, computer vision, NLP, deep learning

Deep Learning

4

MONTHS 8-10

Transformers, LLMs, RAG, fine-tuning, evals

LLMs & Transformers

5

MONTHS 11-12

Agents, multi-agent systems, tool use, memory

Agentic AI

6

MONTHS 13-18

MLOps, LLMOps, cloud, governance, final platform

Enterprise

Foundation



MODULE 1

Python for AI

Core programming & data libraries

Beginner

Duration: 4–6 weeks

WHAT TO LEARN

- Variables, loops, functions, OOP
- List/dict comprehensions
- File I/O & error handling
- Virtual environments & pip

LIBRARIES & TOOLS

NumPy

Pandas

Matplotlib

Seaborn

Jupyter

GRANULAR LEARNING BREAKDOWN

Variables, loops, functions, OOP

variables and primitive types

if/else and loop control

function parameters and return values

classes, objects and simple inheritance

List/dict comprehensions

list comprehensions

dict/set comprehensions

filtering and mapping patterns

readability vs cleverness tradeoffs

File I/O & error handling

read/write text and CSV files

path handling

try/except/finally

custom error messages and logging

Virtual environments & pip

venv creation

requirements files

dependency pinning

project isolation

LEARNING OUTCOMES

- Write clean Python modules instead of only notebooks
- Use virtual environments and package dependencies safely
- Read/write data files and handle errors predictably

HANDS-ON LABS

- Refactor a notebook into a CLI script
- Add logging and exception handling to a data loader
- Write pytest tests for 5 utility functions

DATASETS / PRACTICE INPUTS

SUCCESS RUBRIC

- Kaggle sales data
- CSV/JSON files from data.gov
- synthetic messy customer data

- CLI runs from a fresh environment
- Code has functions, tests, and README
- Outputs are reproducible from raw input

MODULE PROJECT

Exploratory Sales Dashboard

Load a real CSV dataset (Kaggle), clean with Pandas, analyze trends, and produce a full Matplotlib/Seaborn report with 10+ charts and automated insights.

Enterprise habit: Start using professional habits early: README, reproducible environment, testable scripts, and clear folder structure.



MODULE 2

Math for AI

Linear algebra, calculus & stats

Beginner

Duration: 6–8 weeks

WHAT TO LEARN

- Vectors, matrices, dot products
- Derivatives & gradients
- Probability & Bayes' theorem
- Descriptive & inferential statistics

LIBRARIES & TOOLS

NumPy

SciPy

SymPy

Statsmodels

GRANULAR LEARNING BREAKDOWN

Vectors, matrices, dot products

vector notation

matrix multiplication

dot product geometry

cosine similarity

Derivatives & gradients

slope intuition

partial derivatives

chain rule

gradient descent updates

Probability & Bayes' theorem

events and conditional probability

Bayes rule

distributions

uncertainty in predictions

Descriptive & inferential statistics

mean/median/std

sampling

confidence intervals

hypothesis tests

LEARNING OUTCOMES

- Explain dot product, gradients, probability and loss functions intuitively
- Implement gradient descent from scratch
- Connect math notation to working NumPy code

HANDS-ON LABS

- Visualize vector projections
- Derive and code MSE gradient
- Simulate Bayes rule with synthetic data

DATASETS / PRACTICE INPUTS

- synthetic regression data
- Iris numeric features
- Boston/California housing style data

SUCCESS RUBRIC

- Loss decreases correctly
- Manual implementation matches sklearn within tolerance
- Math derivation is included in project notes

MODULE PROJECT

From-Scratch Linear Regression

Implement gradient descent by hand with NumPy. Verify results match Scikit-learn. Plot loss curves, visualize decision boundary, and write a mathematical derivation doc.

Enterprise habit: Start using professional habits early: README, reproducible environment, testable scripts, and clear folder structure.



MODULE 3

Data Science & Pandas

EDA, wrangling, pipelines

Beginner

Duration: 3–4 weeks

WHAT TO LEARN

- DataFrame operations (merge, groupby, pivot)
- Handling missing values & outliers
- Feature engineering basics
- Train/test splits & cross-validation

LIBRARIES & TOOLS

Pandas

Scikit-learn

Missingno

Pandas-Profiling

Plotly

GRANULAR LEARNING BREAKDOWN

DataFrame operations (merge, groupby, pivot)

select/filter/sort

groupby aggregations

joins and keys

pivot tables

Handling missing values & outliers

missingness patterns

imputation strategies

IQR/z-score checks

outlier impact on models

Feature engineering basics

numeric transformations

categorical encoding

date/time features

leakage checks

Train/test splits & cross-validation

holdout split

stratified split

k-fold CV

time-aware validation

LEARNING OUTCOMES

- Clean messy tables and detect data quality issues
- Use groupby, merge, pivot and time series operations
- Create features without leakage

HANDS-ON LABS

- Profile missingness and outliers
- Join 3 related tables
- Create train/test split after leakage review

DATASETS / PRACTICE INPUTS

- Airbnb listings
- Titanic
- NYC taxi samples

SUCCESS RUBRIC

- Data quality report exists
- Feature logic is documented
- Charts answer real business questions

MODULE PROJECT

End-to-End EDA Report

Using a messy real-world dataset (e.g. Titanic, Airbnb), perform full cleaning, outlier removal, feature creation, and auto-generate a Pandas Profiling HTML report with key findings.

Enterprise habit: Start using professional habits early: README, reproducible environment, testable scripts, and clear folder structure.

Machine Learning



MODULE 4

ML Fundamentals

Supervised & unsupervised learning

Beginner

Duration: 6–8 weeks

WHAT TO LEARN

- Linear & logistic regression
- Decision trees & random forests
- K-means & DBSCAN clustering
- Model evaluation: accuracy, F1, AUC

LIBRARIES & TOOLS

Scikit-learn

Pandas

NumPy

Matplotlib

Imbalanced-learn

GRANULAR LEARNING BREAKDOWN

Linear & logistic regression

linear model assumptions

loss functions

classification threshold

regularization

Decision trees & random forests

tree splits

overfitting

bagging

feature importance

K-means & DBSCAN clustering

distance metrics

choosing k

density-based clusters

cluster validation

Model evaluation: accuracy, F1, AUC

confusion matrix

precision/recall

ROC-AUC

business metric selection

LEARNING OUTCOMES

- Train baseline models and compare them fairly
- Choose metrics based on business risk
- Use train/validation/test splits correctly

HANDS-ON LABS

- Build regression and classification baselines
- Compare confusion matrix vs ROC-AUC vs F1
- Save and reload model with joblib

DATASETS / PRACTICE INPUTS

- Telco churn
- credit risk

SUCCESS RUBRIC

- Baseline and tuned model compared
- Leakage checklist completed

- Titanic

- Model has a simple demo UI or API

MODULE PROJECT

Customer Churn Predictor

Full pipeline: load telco churn data → EDA → feature engineering → train 5 Scikit-learn models → compare ROC curves → export best model with joblib. Include a Streamlit demo UI.

Enterprise habit: Start using professional habits early: README, reproducible environment, testable scripts, and clear folder structure.



MODULE 5

Advanced ML & Tuning

Ensembles, pipelines & optimization

Intermediate

Duration: 6–8 weeks

WHAT TO LEARN

- XGBoost, LightGBM, CatBoost
- Hyperparameter tuning with Optuna
- Scikit-learn Pipeline & ColumnTransformer
- SHAP & LIME explainability

LIBRARIES & TOOLS

XGBoost

LightGBM

Optuna

SHAP

Scikit-learn

Feature-engine

GRANULAR LEARNING BREAKDOWN

XGBoost, LightGBM, CatBoost

boosting intuition

tree depth and learning rate

categorical handling

overfitting controls

Hyperparameter tuning with Optuna

search spaces

objective functions

pruning

study tracking

Scikit-learn Pipeline & ColumnTransformer

preprocessing pipelines

numeric/categorical branches

fit vs transform

serialization

SHAP & LIME explainability

global explanations

local explanations

feature attribution limits

stakeholder reporting

LEARNING OUTCOMES

- Build robust sklearn pipelines
- Tune models without overfitting validation data

HANDS-ON LABS

- Create ColumnTransformer pipeline
- Run Optuna study
- Generate SHAP plots for top features

- Explain model behavior with SHAP/LIME

DATASETS / PRACTICE INPUTS

- Kaggle House Prices
- Home Credit
- insurance claim severity

SUCCESS RUBRIC

- Search space is justified
- Leaderboard/holdout improvement shown
- Explanation report is understandable

MODULE PROJECT

House Price Ensemble

Kaggle House Prices dataset. Stack XGBoost + LightGBM + Ridge using Scikit-learn VotingRegressor. Tune with Optuna (200 trials). Generate a full SHAP explanation report. Achieve top-20% leaderboard score.

Enterprise habit: Move from notebooks to repeatable pipelines with validation, metrics, and versioned artifacts.



MODULE 6

ML Theory & Research

Statistics, Bayesian & causality

Advanced

Duration: Ongoing

WHAT TO LEARN

- PAC learning & VC dimension
- Kernel methods & SVMs
- Gaussian processes
- Causal inference & do-calculus

LIBRARIES & TOOLS

Scikit-learn

GPy

DoWhy

PyMC

Statsmodels

GRANULAR LEARNING BREAKDOWN

PAC learning & VC dimension

core concept: PAC learning & VC dimension

small implementation exercise

common failure mode

mini-checkpoint and notes

Kernel methods & SVMs

core concept: Kernel methods & SVMs

small implementation exercise

common failure mode

mini-checkpoint and notes

Gaussian processes

core concept: Gaussian processes

small implementation exercise

common failure mode

mini-checkpoint and notes

Causal inference & do-calculus

core concept: Causal inference & do-calculus

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply ML Theory & Research concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal ML Theory & Research prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- domain-specific corpora
- research paper datasets
- synthetic evaluation sets

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Causal Impact Study

Use DoWhy to model a treatment effect (A/B test or observational study). Compare naive correlation vs causal estimate. Validate with placebo tests and refutation methods. Write a research-style report.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.

Deep Learning



MODULE 7

Deep Learning Basics

Neural nets, backprop & PyTorch

Intermediate

Duration: 6–8 weeks

WHAT TO LEARN

- Perceptrons & multilayer networks
- Backpropagation step-by-step
- Batch norm, dropout, weight init
- Learning rate schedulers

LIBRARIES & TOOLS

PyTorch

TorchMetrics

TensorBoard

Torchvision

GRANULAR LEARNING BREAKDOWN

Perceptrons & multilayer networks

neurons and activations

layers

forward pass

capacity and nonlinearity

Backpropagation step-by-step

computational graphs

loss gradients

autograd

manual gradient checks

Batch norm, dropout, weight init

stabilizing training

regularization

initialization schemes

train vs eval mode

Learning rate schedulers

LR choice

warmup

step/cosine schedules

early stopping

LEARNING OUTCOMES

- Build PyTorch datasets, dataloaders and training loops
- Diagnose overfitting and underfitting
- Use checkpoints and TensorBoard

HANDS-ON LABS

- Implement MLP then CNN
- Add dropout/batch norm comparison
- Resume training from checkpoint

DATASETS / PRACTICE INPUTS

- MNIST
- Fashion-MNIST
- CIFAR-10

SUCCESS RUBRIC

- Training is reproducible
- Curves and confusion matrix are included

- Model export path is tested

MODULE PROJECT

MNIST to CIFAR-10 Classifier

Build a custom CNN in PyTorch from scratch. Train on CIFAR-10, track loss/accuracy in TensorBoard, implement early stopping, achieve >90% val accuracy, and export to TorchScript.

Enterprise habit: Move from notebooks to repeatable pipelines with validation, metrics, and versioned artifacts.



MODULE 8

Computer Vision

CNNs, detection & segmentation

Intermediate

Duration: 6–8 weeks

WHAT TO LEARN

- ResNet, EfficientNet, ViT
- Transfer learning & fine-tuning
- Object detection: YOLO, DETR
- Albumentations augmentation

LIBRARIES & TOOLS

PyTorch

Torchvision

Ultralytics (YOLO)

Albumentations

OpenCV

Timm

GRANULAR LEARNING BREAKDOWN

ResNet, EfficientNet, ViT

residual connections

compound scaling

patch embeddings

model selection

Transfer learning & fine-tuning

frozen backbone

classifier heads

layer unfreezing

data augmentation

Object detection: YOLO, DETR

bounding boxes

IoU

mAP

real-time inference tradeoffs

Albumentations augmentation

image transforms

label-safe transforms

augmentation policies

validation without augmentation

LEARNING OUTCOMES

- Apply Computer Vision concepts in a working project
- Explain the key design choices and tradeoffs

HANDS-ON LABS

- Build a minimal Computer Vision prototype
- Add tests, metrics, and error analysis

- Convert experiments into a reproducible artifact

- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- Kaggle competitions
- OpenML datasets
- Hugging Face Datasets

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Real-Time Object Detection App

Fine-tune YOLOv8 on a custom dataset (annotate with Label Studio). Build a FastAPI endpoint that accepts images and returns bounding boxes. Wrap in a Gradio UI with live webcam support.

Enterprise habit: Move from notebooks to repeatable pipelines with validation, metrics, and versioned artifacts.



MODULE 9

NLP & Sequence Models

Embeddings, RNNs & attention

Intermediate

Duration: 6–8 weeks

WHAT TO LEARN

- Word2Vec, GloVe, FastText
- LSTM & GRU architectures
- Attention mechanism internals
- Tokenization: BPE, WordPiece

LIBRARIES & TOOLS

PyTorch HuggingFace Transformers

NLTK SpaCy Gensim

Datasets (HF)

GRANULAR LEARNING BREAKDOWN

Word2Vec, GloVe, FastText

distributional semantics

embedding training

subword embeddings

similarity search

LSTM & GRU architectures

sequence state

gates

vanishing gradients

sequence classification

Attention mechanism internals

query/key/value

attention weights

masking

interpretation limits

Tokenization: BPE, WordPiece

subword vocabulary

unknown tokens

token length

cost implications

LEARNING OUTCOMES

- Apply NLP & Sequence Models concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal NLP & Sequence Models prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- Kaggle competitions
- OpenML datasets
- Hugging Face Datasets

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Sentiment Analysis API

Fine-tune BERT on a domain-specific dataset (e.g. financial news). Expose via FastAPI. Evaluate with macro-F1, confusion matrix, error analysis. Compare against LSTM baseline. Deploy to HuggingFace Spaces.

Enterprise habit: Move from notebooks to repeatable pipelines with validation, metrics, and versioned artifacts.

LLMs & Transformers



MODULE 10

Transformer Architecture

Self-attention, scaling laws

Advanced

Duration: 4–6 weeks

WHAT TO LEARN

- Scaled dot-product attention
- Multi-head attention & FFN
- Positional encodings: RoPE, ALiBi
- GPT vs BERT vs T5 tradeoffs

LIBRARIES & TOOLS

PyTorch

HuggingFace Transformers

Einops

Flash-Attention

Tiktoken

GRANULAR LEARNING BREAKDOWN

Scaled dot-product attention

Q/K/V matrices

scaling factor

softmax weights

attention masking

Multi-head attention & FFN

parallel attention heads

projection layers

feed-forward blocks

residual + layer norm

Positional encodings: RoPE, ALiBi

why position matters

absolute vs relative position

RoPE intuition

long-context tradeoffs

GPT vs BERT vs T5 tradeoffs

decoder-only

encoder-only

encoder-decoder

task fit

LEARNING OUTCOMES

- Apply Transformer Architecture concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal Transformer Architecture prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- domain-specific corpora
- research paper datasets

SUCCESS RUBRIC

- Project runs from a clean setup

- synthetic evaluation sets

- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Mini-GPT from Scratch

Implement a character-level GPT (Karpathy-style) in PyTorch. Train on Shakespeare or code corpus. Visualize attention weights with BertViz. Add top-k/top-p sampling. Document every architectural decision.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.



MODULE 11

Fine-tuning & Optimization

LoRA, PEFT, quantization

Advanced

Duration: 4–6 weeks

WHAT TO LEARN

- LoRA & QLoRA fine-tuning
- GPTQ & GGUF quantization
- Knowledge distillation
- Inference optimization (vLLM, TGI)

LIBRARIES & TOOLS

PEFT

TRL

BitsAndBytes

vLLM

LitGPT

AutoGPTQ

GRANULAR LEARNING BREAKDOWN

LoRA & QLoRA fine-tuning

adapter concept

rank and alpha

quantized training

merge vs serve adapters

GPTQ & GGUF quantization

weight quantization

quality vs memory

local inference formats

hardware constraints

Knowledge distillation

teacher/student models

soft labels

compression

evaluation

Inference optimization (vLLM, TGI)

batching

KV cache

throughput vs latency

serving metrics

LEARNING OUTCOMES

- Apply Fine-tuning & Optimization concepts in a working project

HANDS-ON LABS

- Build a minimal Fine-tuning & Optimization prototype
- Add tests, metrics, and error analysis

- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- domain-specific corpora
- research paper datasets
- synthetic evaluation sets

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Domain-Specific LLM Assistant

QLoRA fine-tune Llama 3 or Mistral on a custom dataset (medical Q&A, legal docs, or code). Evaluate with ROUGE, BLEU, and human preference scores. Serve with vLLM. Compare base vs fine-tuned with MMLU subset.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.



MODULE 12

Prompt Engineering & RAG

CoT, RAG, evals, structured output

Advanced

Duration: 2–4 weeks

WHAT TO LEARN

- Zero-shot, few-shot, chain-of-thought
- RAG: chunking, retrieval, reranking
- Structured outputs & tool calling
- LLM eval frameworks (RAGAS, DeepEval)

LIBRARIES & TOOLS

LangChain

LlamaIndex

FAISS

ChromaDB

Cohere Rerank

RAGAS

GRANULAR LEARNING BREAKDOWN

Zero-shot, few-shot, chain-of-thought

instruction clarity

examples

reasoning prompts

when not to expose reasoning

RAG: chunking, retrieval, reranking

chunk size

metadata

embedding retrieval

cross-encoder rerankers

Structured outputs & tool calling

JSON schemas

function definitions

validation

retry on invalid output

LLM eval frameworks (RAGAS, DeepEval)

faithfulness

answer relevance

context precision

regression tests

LEARNING OUTCOMES

- Design prompts with structured outputs
- Build retrieval pipelines with chunking and reranking
- Evaluate answer faithfulness and retrieval quality

HANDS-ON LABS

- Compare chunk sizes
- Add metadata filters
- Create 50-question eval set

DATASETS / PRACTICE INPUTS

- company handbook PDFs
- research papers
- product documentation

SUCCESS RUBRIC

- Answers include citations
- Faithfulness and relevance are measured
- Failure cases are documented

MODULE PROJECT

RAG Knowledge Base Q&A

Ingest a 100+ page PDF corpus (e.g. company docs, research papers). Build a hybrid BM25 + dense retrieval pipeline. Add a reranker. Evaluate faithfulness and answer relevance with RAGAS. Deploy as a chat UI.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.

Agentic AI



MODULE 13

AI Agents

Tool use, planning & memory

Advanced

Duration: 4–6 weeks

WHAT TO LEARN

- ReAct & Reflexion patterns
- Function / tool calling APIs
- Short-term vs long-term memory
- LangGraph stateful agents

LIBRARIES & TOOLS

LangChain

LangGraph

OpenAI

Anthropic SDK

Mem0

Tavily

GRANULAR LEARNING BREAKDOWN

ReAct & Reflexion patterns

reason-act-observe loop

reflection

bounded iterations

failure recovery

Function / tool calling APIs

tool schema design

input validation

tool result handling

security boundaries

Short-term vs long-term memory

conversation state

summaries

vector memory

forgetting policies

LangGraph stateful agents

state schema

nodes and edges

conditional routing

checkpointing

LEARNING OUTCOMES

- Design controlled tool-using workflows
- Separate planning, acting, memory and approval
- Avoid unsafe autonomous loops

HANDS-ON LABS

- Build a calculator/search/file tool
- Add human approval before write actions
- Trace every tool call

DATASETS / PRACTICE INPUTS

- web research tasks
- local documents
- mock CRM/ticket data

SUCCESS RUBRIC

- Agent has bounded steps
- Tool inputs/outputs are validated

- Trace shows why each action happened

MODULE PROJECT

Autonomous Research Agent

Build a ReAct agent that accepts a research question, searches the web (Tavily), reads pages, stores findings in a vector memory (Mem0), and produces a structured markdown report. Add human-in-the-loop approval step.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.



MODULE 14

Multi-Agent Systems

Orchestration & collaboration

Advanced

Duration: 4–6 weeks

WHAT TO LEARN

- Agent roles & specialization
- AutoGen / CrewAI patterns
- Inter-agent messaging
- Supervisor + worker hierarchies

LIBRARIES & TOOLS

CrewAI

AutoGen

LangGraph

Pydantic AI

Redis (message queue)

GRANULAR LEARNING BREAKDOWN

Agent roles & specialization

planner role

researcher role

critic role

handoffs

AutoGen / CrewAI patterns

multi-agent conversations

supervisor patterns

task delegation

termination rules

Inter-agent messaging

message contracts

shared state

conflict handling

traceability

Supervisor + worker hierarchies

orchestrator design

worker tools

approval gates

fallback routes

LEARNING OUTCOMES

- Apply Multi-Agent Systems concepts in a working project
- Explain the key design choices and tradeoffs

HANDS-ON LABS

- Build a minimal Multi-Agent Systems prototype
- Add tests, metrics, and error analysis

- Convert experiments into a reproducible artifact

- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- domain-specific corpora
- research paper datasets
- synthetic evaluation sets

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

AI Startup Analysis Team

CrewAI team of 4 agents: Market Researcher, Financial Analyst, Technical Reviewer, Report Writer. Given a startup name, they collaboratively produce a 5-page VC-style due diligence report with sourced data.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.



MODULE 15

Agentic Architecture

Production pipelines at scale

Enterprise

Duration: Ongoing

WHAT TO LEARN

- Stateful agent design with LangGraph
- Human-in-the-loop patterns
- Fault tolerance & retry strategies
- Observability with LangSmith

LIBRARIES & TOOLS

LangSmith

LangGraph

Celery

Redis

OpenTelemetry

Sentry

GRANULAR LEARNING BREAKDOWN

Stateful agent design with LangGraph

core concept: Stateful agent design with LangGraph

small implementation exercise

common failure mode

mini-checkpoint and notes

Human-in-the-loop patterns

core concept: Human-in-the-loop patterns

small implementation exercise

common failure mode

mini-checkpoint and notes

Fault tolerance & retry strategies

core concept: Fault tolerance & retry strategies

small implementation exercise

common failure mode

mini-checkpoint and notes

Observability with LangSmith

core concept: Observability with LangSmith

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply Agentic Architecture concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal Agentic Architecture prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Production Agent with Observability

Harden the research agent: add LangSmith tracing, retry logic, fallback LLM providers, rate-limit handling, structured logging. Deploy as a scalable Celery worker. Monitor latency and token costs in a Grafana dashboard.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.

Enterprise & MLOps



MODULE 16

MLOps

Deployment, monitoring & CI/CD

Advanced

Duration: 6–8 weeks

WHAT TO LEARN

- MLflow experiment tracking
- Model packaging with FastAPI + Docker
- DVC for data versioning
- GitHub Actions ML pipeline

LIBRARIES & TOOLS

MLflow

DVC

FastAPI

Docker

Pytest

Great Expectations

GRANULAR LEARNING BREAKDOWN

MLflow experiment tracking

runs and params

metrics and artifacts

model registry

comparison

Model packaging with FastAPI + Docker

predict endpoint

schema validation

container build

health checks

DVC for data versioning

data remotes

pipeline stages

reproducibility

large-file workflow

GitHub Actions ML pipeline

test jobs

lint/type checks

build image

deployment gates

LEARNING OUTCOMES

- Package models behind APIs
- Track experiments and register artifacts
- Automate testing and deployment checks

HANDS-ON LABS

- Create FastAPI predict endpoint
- Track model in MLflow
- Build Docker image and CI test

DATASETS / PRACTICE INPUTS

- any prior ML project dataset
- drifted synthetic data

SUCCESS RUBRIC

- API has health and prediction tests
- Model version is logged

- batch scoring sample

- Rollback plan exists

MODULE PROJECT

Automated ML Pipeline

Full MLOps pipeline: DVC-versioned data → Scikit-learn/PyTorch model → MLflow tracking → Docker image → FastAPI endpoint → GitHub Actions CI that runs tests and auto-deploys on main merge. Add data validation with Great Expectations.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.



MODULE 17

Cloud AI Platforms

AWS, GCP, Azure ML services

Enterprise

Duration: 4–6 weeks

WHAT TO LEARN

- SageMaker training jobs & endpoints
- Vertex AI Pipelines
- Azure ML + promptflow
- Cost optimization & spot instances

LIBRARIES & TOOLS

boto3

SageMaker SDK

Vertex AI SDK

Terraform

Pulumi

GRANULAR LEARNING BREAKDOWN

SageMaker training jobs & endpoints

core concept: SageMaker training jobs & endpoints

small implementation exercise

common failure mode

mini-checkpoint and notes

Vertex AI Pipelines

core concept: Vertex AI Pipelines

small implementation exercise

common failure mode

mini-checkpoint and notes

Azure ML + promptflow

core concept: Azure ML + promptflow

small implementation exercise

common failure mode

mini-checkpoint and notes

Cost optimization & spot instances

core concept: Cost optimization & spot instances

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

HANDS-ON LABS

- Apply Cloud AI Platforms concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

- Build a minimal Cloud AI Platforms prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

SageMaker End-to-End Deploy

Train an XGBoost model on SageMaker, register in Model Registry, deploy as a real-time endpoint with auto-scaling, set up CloudWatch alarms for drift, and provision all infrastructure with Terraform.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.



MODULE 18

AI Safety & Governance

Responsible AI, fairness, compliance

Enterprise

Duration: Ongoing

WHAT TO LEARN

- Bias auditing: Fairlearn, AIF360
- Explainability: SHAP, LIME
- EU AI Act & model cards
- Red-teaming & adversarial prompts

LIBRARIES & TOOLS

Fairlearn AIF360 SHAP Alibi
Guardrails AI Garak

GRANULAR LEARNING BREAKDOWN

Bias auditing: Fairlearn, AIF360

sensitive attributes

group metrics

mitigation

audit reporting

Explainability: SHAP, LIME

model-agnostic explanations

tree explanations

local examples

limitations

EU AI Act & model cards

risk classification

model documentation

intended use

limitations

Red-teaming & adversarial prompts

attack prompts

jailbreak tests

policy checks

mitigation backlog

LEARNING OUTCOMES

- Apply AI Safety & Governance concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal AI Safety & Governance prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Model Fairness Audit Report

Take a production-grade model (loan approval, hiring), run a full Fairlearn + SHAP audit across demographic groups, identify disparate impact, apply mitigation (reweighing, threshold adjustment), and publish a model card.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.



MODULE 19

LLMOps

Running LLMs in production

Enterprise

Duration: Ongoing

WHAT TO LEARN

- Prompt versioning with PromptLayer
- A/B testing LLM responses
- Guardrails & content filtering
- Token cost & latency SLAs

LIBRARIES & TOOLS

PromptLayer

LangSmith

Guardrails AI

Helicone

Langfuse

OpenCost

GRANULAR LEARNING BREAKDOWN

Prompt versioning with PromptLayer

prompt IDs

version history

rollback

experiment comparison

A/B testing LLM responses

variant design

quality metrics

human preference

statistical confidence

Guardrails & content filtering

input validation

output validation

policy filters

safe fallback

Token cost & latency SLAs

token accounting

latency budgets

provider routing

cost dashboards

LEARNING OUTCOMES

- Apply LLMOps concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal LLMOps prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

LLM Gateway with Analytics

Build an LLM proxy gateway (using LiteLLM) that routes to multiple providers, logs every request to Langfuse, enforces guardrails, tracks cost per team with OpenCost, and exposes a Grafana dashboard. A/B test two prompt versions automatically.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.

Missing Enterprise Foundations Added



MODULE 20

NumPy, SciPy & Scientific Computing

Array thinking, vectorization, numerical stability

Beginner

Duration: 2-3 weeks

WHAT TO LEARN

- Broadcasting, vectorization, memory layout
- Numerical precision and floating point traps
- SciPy optimization, sparse matrices, signal/stat modules
- Reusable notebook-to-package workflow

LIBRARIES & TOOLS

NumPy

SciPy

Jupyter

pytest

GRANULAR LEARNING BREAKDOWN

Broadcasting, vectorization, memory layout

core concept: Broadcasting, vectorization, memory layout

small implementation exercise

common failure mode

mini-checkpoint and notes

Numerical precision and floating point traps

core concept: Numerical precision and floating point traps

small implementation exercise

common failure mode

mini-checkpoint and notes

SciPy optimization, sparse matrices, signal/stat modules

core concept: SciPy optimization, sparse matrices, signal/stat modules

small implementation exercise

common failure mode

mini-checkpoint and notes

Reusable notebook-to-package workflow

core concept: Reusable notebook-to-package workflow

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply NumPy, SciPy & Scientific Computing concepts in a working project

HANDS-ON LABS

- Build a minimal NumPy, SciPy & Scientific Computing prototype
- Add tests, metrics, and error analysis

- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- Kaggle beginner datasets
- UCI Machine Learning Repository
- public CSV exports

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Scientific Computing Lab

Implement vectorized distance search, sparse matrix operations, and an optimization routine; benchmark loop vs vectorized implementations and publish a performance note.

Enterprise habit: Start using professional habits early: README, reproducible environment, testable scripts, and clear folder structure.



MODULE 21

SQL & Data Engineering for AI

Reliable data pipelines before model pipelines

Intermediate

Duration: 4-6 weeks

WHAT TO LEARN

- SQL joins, windows, CTEs, indexing basics
- Batch pipelines, orchestration, data contracts
- Feature stores and offline/online consistency
- Data validation, lineage, and schema evolution

LIBRARIES & TOOLS

PostgreSQL

DuckDB

Airflow

dbt

Great Expectations

Feast

GRANULAR LEARNING BREAKDOWN

SQL joins, windows, CTEs, indexing basics

core concept: SQL joins, windows, CTEs, indexing basics

small implementation exercise

common failure mode

mini-checkpoint and notes

Batch pipelines, orchestration, data contracts

core concept: Batch pipelines, orchestration, data contracts

small implementation exercise

common failure mode

mini-checkpoint and notes

Feature stores and offline/online consistency

core concept: Feature stores and offline/online consistency

small implementation exercise

common failure mode

mini-checkpoint and notes

Data validation, lineage, and schema evolution

core concept: Data validation, lineage, and schema evolution

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply SQL & Data Engineering for AI concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal SQL & Data Engineering for AI prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- Kaggle competitions
- OpenML datasets
- Hugging Face Datasets

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Production Feature Pipeline

Build a SQL + Python feature pipeline with validation gates, reproducible snapshots, and a feature table consumed by both sklearn training and FastAPI inference.

Enterprise habit: Move from notebooks to repeatable pipelines with validation, metrics, and versioned artifacts.



MODULE 22

AI Evaluation & Testing

Regression tests for models, prompts, RAG and agents

Advanced

Duration: 3-4 weeks

WHAT TO LEARN

- Golden datasets and offline evals
- Prompt regression tests and adversarial cases
- RAG faithfulness, answer relevance, retrieval recall
- Agent trajectory evaluation and tool-call correctness

LIBRARIES & TOOLS

OpenAI Evals

DeepEval

RAGAS

pytest

LangSmith

MLflow

GRANULAR LEARNING BREAKDOWN

Golden datasets and offline evals

core concept: Golden datasets and offline evals

small implementation exercise

common failure mode

mini-checkpoint and notes

Prompt regression tests and adversarial cases

core concept: Prompt regression tests and adversarial cases

small implementation exercise

common failure mode

mini-checkpoint and notes

RAG faithfulness, answer relevance, retrieval recall

core concept: RAG faithfulness, answer relevance, retrieval recall

small implementation exercise

common failure mode

mini-checkpoint and notes

Agent trajectory evaluation and tool-call correctness

core concept: Agent trajectory evaluation and tool-call correctness

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Build eval sets for ML, RAG and agents
- Create quality gates for deployment
- Track regressions over time

HANDS-ON LABS

- Write prompt regression tests
- Score RAG faithfulness
- Evaluate agent tool-call accuracy

DATASETS / PRACTICE INPUTS

- golden Q&A pairs
- adversarial prompts
- known-bad retrieval examples

SUCCESS RUBRIC

- CI can fail on eval regression
- Metrics have thresholds
- Human review notes are captured

MODULE PROJECT

AI Quality Gate

Create an automated eval suite that blocks deployment when model quality, hallucination rate, latency, or cost exceeds defined thresholds.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.



MODULE 23

Observability & Reliability

Enterprise

Operate AI systems with traces, alerts and runbooks

Duration: 3-4 weeks

WHAT TO LEARN

- Structured logs, traces, metrics and audit events
- Latency, cost, token usage and error budgets
- Data drift, model drift and feedback loops
- Incident response and rollback playbooks

LIBRARIES & TOOLS

OpenTelemetry

Prometheus

Grafana

Sentry

Langfuse

Helicone

GRANULAR LEARNING BREAKDOWN

Structured logs, traces, metrics and audit events

core concept: Structured logs, traces, metrics and audit events

small implementation exercise

common failure mode

mini-checkpoint and notes

Latency, cost, token usage and error budgets

core concept: Latency, cost, token usage and error budgets

small implementation exercise

common failure mode

mini-checkpoint and notes

Data drift, model drift and feedback loops

core concept: Data drift, model drift and feedback loops

small implementation exercise

common failure mode

mini-checkpoint and notes

Incident response and rollback playbooks

core concept: Incident response and rollback playbooks

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

HANDS-ON LABS

- Apply Observability & Reliability concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

- Build a minimal Observability & Reliability prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

AI Observability Dashboard

Instrument an LLM/RAG API with traces, token/cost tracking, quality scores, error alerts, and a runbook for degraded retrieval or model outages.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.



MODULE 24

MCP, Tools & Integration Protocols

Connect agents safely to enterprise tools

Enterprise

Duration: 2-3 weeks

WHAT TO LEARN

- Tool schemas, authentication, authorization
- Model Context Protocol server concepts
- Connector safety, least privilege, approval gates
- Tool result validation and sandboxing

LIBRARIES & TOOLS

MCP

FastAPI

Pydantic

OAuth2

OpenAPI

GRANULAR LEARNING BREAKDOWN

Tool schemas, authentication, authorization

core concept: Tool schemas, authentication, authorization

small implementation exercise

common failure mode

mini-checkpoint and notes

Model Context Protocol server concepts

core concept: Model Context Protocol server concepts

small implementation exercise

common failure mode

mini-checkpoint and notes

Connector safety, least privilege, approval gates

core concept: Connector safety, least privilege, approval gates

small implementation exercise

common failure mode

mini-checkpoint and notes

Tool result validation and sandboxing

core concept: Tool result validation and sandboxing

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply MCP, Tools & Integration Protocols concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal MCP, Tools & Integration Protocols prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Secure Tool Server

Build a small MCP-style tool server that exposes read-only database/document tools with auth, input validation, logging, and human approval for sensitive actions.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.



Multimodal AI

Vision, documents, audio and speech with LLMs

Duration: 4-6 weeks

WHAT TO LEARN

- Vision-language models and document understanding
- OCR, layout extraction and multimodal retrieval
- Speech-to-text, text-to-speech and voice agents
- Evaluation for image, document and audio outputs

LIBRARIES & TOOLS

OpenAI Vision/Audio

Whisper

Tesseract

LayoutLM

CLIP

Pydub

GRANULAR LEARNING BREAKDOWN

Vision-language models and document understanding

core concept: Vision-language models and document understanding

small implementation exercise

common failure mode

mini-checkpoint and notes

OCR, layout extraction and multimodal retrieval

core concept: OCR, layout extraction and multimodal retrieval

small implementation exercise

common failure mode

mini-checkpoint and notes

Speech-to-text, text-to-speech and voice agents

core concept: Speech-to-text, text-to-speech and voice agents

small implementation exercise

common failure mode

mini-checkpoint and notes

Evaluation for image, document and audio outputs

core concept: Evaluation for image, document and audio outputs

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply Multimodal AI concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal Multimodal AI prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

SUCCESS RUBRIC

- domain-specific corpora
- research paper datasets
- synthetic evaluation sets

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Multimodal Document Analyst

Ingest PDFs with charts/images, extract text and layout, answer questions with citations, and optionally generate an audio briefing.

Enterprise habit: Add evaluation, error analysis, failure-mode documentation, and explicit tradeoff decisions.



MODULE 26

Security, Privacy & Compliance

Production controls for sensitive AI workloads

Enterprise

Duration: Ongoing

WHAT TO LEARN

- PII detection, masking and retention policies
- Prompt injection and data exfiltration defenses
- Secrets management and workload identity
- Auditability, access control and compliance review

LIBRARIES & TOOLS

Presidio

Vault

OPA

Guardrails AI

Garak

OWASP LLM Top 10

GRANULAR LEARNING BREAKDOWN

PII detection, masking and retention policies

core concept: PII detection, masking and retention policies

small implementation exercise

common failure mode

mini-checkpoint and notes

Prompt injection and data exfiltration defenses

core concept: Prompt injection and data exfiltration defenses

small implementation exercise

common failure mode

mini-checkpoint and notes

Secrets management and workload identity

core concept: Secrets management and workload identity

small implementation exercise

common failure mode

mini-checkpoint and notes

Auditability, access control and compliance review

core concept: Auditability, access control and compliance review

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Identify sensitive data paths
- Mitigate prompt injection and data exfiltration
- Create audit-ready controls

HANDS-ON LABS

- Run PII detection/redaction
- Create prompt-injection test suite
- Add allowlisted tool permissions

DATASETS / PRACTICE INPUTS

- synthetic PII records
- OWASP-style attack prompts
- mock internal documents

SUCCESS RUBRIC

- Threat model is written
- PII never appears in logs
- Security tests are repeatable

MODULE PROJECT

Secure RAG Review

Threat-model a RAG system, add PII redaction, prompt-injection tests, allowlisted tools, audit logs, and a compliance-ready risk report.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.



MODULE 27

Kubernetes & Platform Engineering

Scale APIs, workers and model services

Enterprise

Duration: 4-6 weeks

WHAT TO LEARN

- Containers, images, registries and runtime limits
- Kubernetes deployments, services, jobs and autoscaling
- GPU scheduling, queue workers and async inference
- Blue/green and canary deployments

LIBRARIES & TOOLS

Docker

Kubernetes

Helm

KServe

Ray Serve

Argo Workflows

GRANULAR LEARNING BREAKDOWN

Containers, images, registries and runtime limits

core concept: Containers, images, registries and runtime limits

small implementation exercise

common failure mode

mini-checkpoint and notes

Kubernetes deployments, services, jobs and autoscaling

core concept: Kubernetes deployments, services, jobs and autoscaling

small implementation exercise

common failure mode

mini-checkpoint and notes

GPU scheduling, queue workers and async inference

core concept: GPU scheduling, queue workers and async inference

small implementation exercise

common failure mode

mini-checkpoint and notes

Blue/green and canary deployments

core concept: Blue/green and canary deployments

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply Kubernetes & Platform Engineering concepts in a working project
- Explain the key design choices and tradeoffs
- Convert experiments into a reproducible artifact

HANDS-ON LABS

- Build a minimal Kubernetes & Platform Engineering prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Scalable AI Service

Deploy a RAG or model inference API with separate worker queues, autoscaling, health checks, canary rollout, and dashboarded SLOs.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational



MODULE 28

Enterprise

AI Product & Portfolio

Turn technical projects into business outcomes

Duration: Ongoing

WHAT TO LEARN

- Problem framing and success metrics
- Cost, latency, quality and risk tradeoffs
- User feedback loops and human workflow design
- Portfolio storytelling, READMEs, model cards and demos

LIBRARIES & TOOLS

ADR

Model Cards

PRDs

User Journey Maps

GRANULAR LEARNING BREAKDOWN

Problem framing and success metrics

core concept: Problem framing and success metrics

small implementation exercise

common failure mode

mini-checkpoint and notes

Cost, latency, quality and risk tradeoffs

core concept: Cost, latency, quality and risk tradeoffs

small implementation exercise

common failure mode

mini-checkpoint and notes

User feedback loops and human workflow design

core concept: User feedback loops and human workflow design

small implementation exercise

common failure mode

mini-checkpoint and notes

Portfolio storytelling, READMEs, model cards and demos

core concept: Portfolio storytelling, READMEs, model cards and demos

small implementation exercise

common failure mode

mini-checkpoint and notes

LEARNING OUTCOMES

- Apply AI Product & Portfolio concepts in a working project
- Explain the key design choices and tradeoffs

HANDS-ON LABS

- Build a minimal AI Product & Portfolio prototype
- Add tests, metrics, and error analysis
- Refactor the work into a reusable project structure

- Convert experiments into a reproducible artifact

DATASETS / PRACTICE INPUTS

- internal-style mock data
- privacy-safe sampled data
- production logs and telemetry samples

SUCCESS RUBRIC

- Project runs from a clean setup
- Metrics and failure cases are documented
- README explains assumptions, limitations, and next steps

MODULE PROJECT

Enterprise AI Case Study

Write a polished case study for your final platform: business problem, architecture, metrics, risks, tradeoffs, screenshots, and roadmap.

Enterprise habit: Design for auditability, security, reliability, cost control, monitoring, and operational ownership.

AutoResearch AI — Enterprise Agentic Intelligence Platform

A production-grade multi-agent system that researches, analyzes, writes reports, and deploys insights end-to-end.



What it does

A user submits a research query. The platform creates a plan, retrieves documents through RAG, analyzes structured data with ML models, writes a report with an LLM, reviews it with a critic agent, and serves results through a FastAPI dashboard with streaming and traceability.

Architecture Layers

UI Layer

Streamlit or React dashboard, FastAPI REST, WebSocket streaming.

Agent Layer

LangGraph state machine, planner, researcher, analyst, writer, critic.

RAG Layer

FAISS or vector DB, hybrid retrieval, reranking, citations.

ML Layer

pandas, sklearn analytics, PyTorch sentiment/classifier model.

LLM Layer

Structured outputs, prompt versions, optional LoRA-tuned writer.

Ops Layer

MLflow, Docker, CI/CD, observability, evals, guardrails.

12-Week Build Plan

Weeks 1-2

Data ingestion, EDA, document parser, vector index.

Weeks 3-4

sklearn analytics model and PyTorch classifier.

Weeks 5-6

RAG pipeline, prompt design, structured outputs, eval set.

Weeks 7-8

Multi-agent orchestration, tools, memory, human approval.

Weeks 9-10

FastAPI backend, dashboard, streaming, auth assumptions.

Weeks 11-12

Docker, CI/CD, MLflow, monitoring, safety review, final demo.

Definition of Done

- Answers include citations, uncertainty notes and tool traces.
- At least three tools are used: retrieval, SQL/data, ML inference or calculator.
- Model, prompt and data versions are logged for every run.
- Offline evals test retrieval quality, answer quality, safety and cost.
- README includes architecture, setup, risks, screenshots, and runbook.

ASSESSMENT

How to Judge Progress

Do not mark a module complete because you watched a course. Mark it complete when the project survives this review checklist.

Concept mastery

Can explain the idea without notes, derive or sketch the mechanism, and identify where it fails.

Implementation

Can build the core workflow from scratch or with standard libraries and keep it reproducible.

Evaluation

Defines the right metrics, creates test/eval examples, analyzes errors, and compares baselines.

Engineering quality

Uses clean structure, tests, configuration, logging, versioning and automation.

Enterprise readiness

Documents security, privacy, observability, cost, rollback and governance assumptions.

Scoring Rule

0

Not attempted

1

Followed tutorial only

2

Built independently with gaps

3

Reproducible, evaluated, documented

4

Production-minded, tested, observable

Promotion rule: move to the next major track only when your average score is at least 3 across concept, implementation and evaluation.

Project Artifact Template

Use this same artifact structure for every module. It makes your portfolio feel like real engineering work, not a folder of disconnected notebooks.

README

Problem, dataset, setup, run commands, screenshots, architecture, metrics and limitations.

Source code

src package, notebooks only for exploration, tests, configs, Makefile or task runner.

Data notes

Data source, schema, cleaning rules, leakage review, privacy considerations.

Evaluation

Baseline, metrics, failure cases, confusion matrix or qualitative examples.

Deployment

API or app, Dockerfile, environment variables, health check and rollback note.

Governance

Model card, risk register, cost estimate, monitoring plan and owner/runbook.

Recommended Repository Layout

```
project-name/  
  README.md  
  pyproject.toml or requirements.txt  
  data/README.md  
  notebooks/01_exploration.ipynb  
  src/  
  tests/  
  configs/  
  scripts/  
  app/ or api/  
  reports/  
  model_card.md  
  risk_register.md
```

CHECKLIST

Enterprise Portfolio Checklist

✓ Every project has a README, setup command, screenshots and metrics.

✓ Every ML project has baseline, validation strategy and error analysis.

✓ Every LLM/RAG project has eval examples and failure cases.

✓ Every deployed service has tests, logs, health check and rollback notes.

✓ Every enterprise project has security, privacy and governance assumptions.

✓ The final capstone has an architecture diagram, model card and runbook.

REFERENCES

Official References

Use official documentation for syntax, APIs and current platform behavior. Libraries move quickly; verify versions before starting each project.

1. **Python Tutorial**

<https://docs.python.org/3/tutorial/>

2. **pandas Getting Started**

https://pandas.pydata.org/docs/getting_started/index.html

3. **scikit-learn User Guide**

https://scikit-learn.org/stable/user_guide.html

4. **PyTorch Tutorials**

<https://docs.pytorch.org/tutorials/>

5. **Hugging Face Transformers**

<https://huggingface.co/docs/transformers/index>

6. **LangChain / LangGraph Docs**

<https://docs.langchain.com/>

7. **OpenAI Agents, Retrieval, Evals, Safety**

<https://platform.openai.com/docs/>

8. **MLflow Documentation**

<https://mlflow.org/docs/latest/index.html>